# Lab 07 — Praktijklab: web → api → db op naam

*Doel: zien wat het standaard rootless-netwerk echt geeft, een netwerk met DNS aanmaken, poorten publiceren, de keten nginx → API → PostgreSQL op naam opbouwen, de DNS van nginx breken om de `502` te begrijpen, en daarna hetzelfde doen in een pod.*

**Vereisten** — Labs 01 tot 06 afgewerkt. `~/labo-docker/05/Dockerfile` en `Api.java` aanwezig (lab 05). Images `alpine`, `nginx:alpine`, `eclipse-temurin:21-jdk`, `eclipse-temurin:21-jre-alpine`; `postgres:16-alpine` wordt opnieuw opgehaald.

**Meegeleverde bestanden** ( `files/` ) - `nginx-proxy.conf` — nginx serveert de frontend en stuurt `/api/` door naar `http://api:8080/`. - `nginx-pod.conf` — hetzelfde naar `http://localhost:8080/`, voor de pod-stap. - `index.html` — de "Angular"-pagina die `/api/actuator/health` aanroept.

De adressen `10.89.0.x` hieronder zijn die van een vers aangemaakt netwerk; de jouwe kunnen een of twee eenheden verschillen.

---

## Stap 0 — De API opnieuw bouwen

```
mkdir -p ~/labo-docker/07 && cd ~/labo-docker/07
cp <pad-naar-het-lab>/files/* .
podman build -q -t api:1.0 ~/labo-docker/05
podman images --format '{{.Repository}}:{{.Tag}} {{.Size}}' | grep api
```

**Observeer** `localhost/api:1.0 209 MB`: de multi-stage van lab 05, in enkele seconden herbouwd dankzij de cache.

---

## Stap 1 — Het standaardnetwerk: pasta

```
podman run -d --name solo alpine sleep 1d
podman inspect solo --format '{{.NetworkSettings.IPAddress}}|{{.HostConfig.NetworkMode}}'
podman exec solo ip -4 -o addr show eth0
ip -4 -o addr show scope global | head -1
podman exec solo cat /etc/resolv.conf
podman exec solo wget -qO- --timeout=5 http://1.1.1.1 | head -c 60 ; echo
podman rm -f -t 0 solo
```

**Observeer** `|pasta` — geen adres, modus `pasta` —, daarna een `eth0` in de container met **hetzelfde adres als je WSL** ( `172.2x.x.x/20` ), de `resolv.conf` van de host, en HTML: het internet werkt.

*Uitleg.* Zonder rootrechten geen bridge: `pasta` bootst de netwerkstack van de host na in de container. Handig voor één container; onbruikbaar om er meerdere met elkaar te laten praten.

---

## Stap 2 — De bridge `podman`: IP's, geen namen

```
podman network ls
podman network inspect podman | grep -E '"(subnet|gateway|dns_enabled)"'
podman run -d --name p1 --network podman alpine sleep 1d
podman run -d --name p2 --network podman alpine sleep 1d
podman inspect p1 --format '{{.NetworkSettings.Networks.podman.IPAddress}}'
podman exec p2 ping -c1 -W1 $(podman inspect p1 --format
'{{.NetworkSettings.Networks.podman.IPAddress}}') | head -2
podman exec p2 ping -c1 -W1 p1
podman rm -f -t 0 p1 p2
```

**Observeer** `"subnet": "10.88.0.0/16"`, `"dns_enabled": false`; `p1` heeft een adres `10.88.0.x`; `ping` op IP antwoordt (`64 bytes from 10.88.0.x`); `ping p1` faalt: `ping: bad address 'p1'`.

*Uitleg.* De bridge verbindt de containers, maar het netwerk `podman` heeft geen DNS. `10.88.0.4` vastleggen in een configuratie zou een tijdbom zijn: het adres verandert bij de volgende aanmaak.

---

## Stap 3 — Een user-defined netwerk met DNS

```
podman network create stack
podman network inspect stack | grep -E '"(name|subnet|gateway|dns_enabled)"'
podman run -d --name db --network stack -e POSTGRES_PASSWORD=secret postgres:16-alpine
podman run -d --name api --network stack -e APP_MESSAGE="hallo van het netwerk stack" api:1.0
sleep 3 ; podman exec api cat /etc/resolv.conf
podman run --rm --network stack alpine sh -c 'getent hosts api db; wget -qO-
http://api:8080/actuator/health; echo; nc -zv db 5432'
podman run --rm --network stack -e PGPASSWORD=secret postgres:16-alpine psql -h db -U postgres
-c 'select version()' | head -3
```

**Observeer** `"subnet": "10.89.0.0/24"`, `"dns_enabled": true`; in `api`, `search dns.podman` en `nameserver 10.89.0.1`; daarna `10.89.0.3 api.dns.podman … api`, `10.89.0.2 db.dns.podman … db`, `{"status":"UP"}`, `db (10.89.0.2:5432) open`, en `PostgreSQL 16.x on x86_64-pc-linux-musl`: een wegwerpclient heeft de database **op naam** bereikt.

```
podman network inspect stack --format '{{range .Containers}}{{.Name}} {{end}}'
```

**Observeer** `api db`: het netwerk kent zijn leden.

*Uitleg.* Het netwerk dat je zelf aanmaakt, start `aardvark-dns` op de gateway. Elke container van het netwerk resolvet de namen van de andere. Het is de enige gezonde manier om diensten te verbinden.

---

## Stap 4 — Een poort publiceren

```
podman image inspect api:1.0 --format '{{.Config.ExposedPorts}}'
podman port api
curl -s --max-time 2 localhost:8080/actuator/health || echo "(onbereikbaar vanaf de host)"
```

**Observeer** `map[8080/tcp:{}]` — de image *declareert* 8080 —, een lege `podman port api`, en `(onbereikbaar vanaf de host)`: `EXPOSE` publiceert niets.

```
podman run -d --name api-pub --network stack -p 18080:8080 api:1.0
sleep 2 ; podman port api-pub ; curl -s localhost:18080/actuator/health ; echo
podman run -d --name api-rand --network stack -P api:1.0
podman port api-rand
podman run -d --name api-lo --network stack -p 127.0.0.1:18081:8080 api:1.0
podman port api-lo
podman ps --format 'table {{.Names}}\t{{.Ports}}'
```

**Observeer** `8080/tcp -> 0.0.0.0:18080` en `{"status":"UP"}`; met `-P` een willekeurige hostpoort (`0.0.0.0:39843`); met `127.0.0.1:` , `127.0.0.1:18081`; en in `ps`, `db 5432/tcp` en `api 8080/tcp` **zonder pijl** (exposed, niet gepubliceerd) tegenover `api-pub 0.0.0.0:18080->8080/tcp`.

```
podman run -d --name api-80 --network stack -p 80:8080 api:1.0 ; echo "exit=$?"
podman ps -a --filter name=api-80 --format '{{.Status}}'
podman rm -f -t 0 api-pub api-rand api-lo api-80
```

**Observeer** `Error: rootlessport cannot expose privileged port 80, you can add 'net.ipv4.ip_unprivileged_port_start=80' to /etc/sysctl.conf (currently 1024), or choose a larger port number (>= 1024): listen tcp 0.0.0.0:80: bind: permission denied`, `exit=126`, en een container die in `Created` blijft.

*Uitleg.* Publiceren is de host vragen om te luisteren voor de container. In rootless ben jij de host, en poort 80 heb je niet.

---

## Stap 5 — De keten web → api → db

```
podman run -d --name web --network stack -p 18080:80 \
  -v ./nginx-proxy.conf:/etc/nginx/conf.d/default.conf:ro \
  -v ./index.html:/usr/share/nginx/html/index.html:ro nginx:alpine
sleep 1 ; curl -s localhost:18080/ | head -3
curl -s localhost:18080/api/actuator/health ; echo
curl -s localhost:18080/api/ ; echo
podman logs web 2>&1 | tail -1
```

**Observeer** de `<!doctype html>` van de frontend, daarna `{"status":"UP"}` en `{"message":"hallo van het netwerk stack","profile":"default"}`: nginx heeft `/api/` doorgestuurd naar `api:8080` en het voorvoegsel weggehaald. In de log is het clientadres dat van `web` zelf (`10.89.0.10` hier: de containers van stap 4 hebben de vorige adressen opgebruikt).

*Uitleg.* Eén gepubliceerde poort, die van nginx. De API en de database zijn alleen op naam bereikbaar, vanuit het netwerk. Open `http://localhost:18080` in je Windows-browser: de pagina roept `/api/actuator/health` aan en toont `{"status":"UP"}` — zonder CORS, want alles komt van dezelfde origin.

---

## Stap 6 — De DNS van nginx breken

```
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
podman inspect api --format '{{.NetworkSettings.Networks.stack.IPAddress}}'
podman stop -t 1 api
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
podman run -d --name indringer --network stack alpine sleep 1d
podman start api ; sleep 2
podman inspect api --format '{{.NetworkSettings.Networks.stack.IPAddress}}'
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
podman logs web 2>&1 | grep -i error | tail -1 | cut -c1-120
```

**Observeer** `200` en `10.89.0.3`; na het stoppen `504`; na de herstart heeft de API een ander adres (`10.89.0.12`: `indringer` nam het vorige) en antwoordt nginx `502`, met `connect() failed (113: Host is unreachable) while connecting to upstream`.

```
podman run --rm --network stack alpine wget -qO- http://api:8080/actuator/health ; echo
podman restart -t 1 web ; sleep 1
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
podman rm -f -t 0 indringer
```

**Observeer** dat de wegwerpcontainer `api` probleemloos bereikt (de DNS is bij), en dat na de `restart` van `web` het antwoord `200` is.

*Uitleg.* nginx legt de IP's van zijn `proxy_pass` vast bij het opstarten. De DNS is niet de schuldige: het is de client die hem niet opnieuw bevraagt. nginx herstarten, of hem een `resolver` geven.

---

## Stap 7 — Twee netwerken en een alias

```
podman network create admin
podman run -d --name mon --network admin alpine sleep 1d
podman exec mon wget -qO- --timeout=2 http://api:8080/actuator/health || echo "api onbereikbaar
vanuit admin"
podman network connect --alias backend admin api
podman exec mon wget -qO- --timeout=2 http://backend:8080/actuator/health ; echo
podman inspect api --format '{{range $k,$v := .NetworkSettings.Networks}}{{$k}}=
{{$v.IPAddress}} {{end}}'
podman network disconnect admin api
podman rm -f -t 0 mon ; podman network rm admin
```

**Observeer** `wget: bad address 'api:8080'` en dan `api onbereikbaar vanuit admin`; na het aansluiten `{"status":"UP"}` via de alias `backend`; en twee adressen: `admin=10.89.1.3 stack=10.89.0.12`.

*Uitleg.* Een netwerk is ook een grens: `mon` zag `api` niet. `api` op `admin` aansluiten geeft haar een interface en een naam op dat netwerk alleen.

---

## Stap 8 — host, none, en de host vanuit een container

```
podman run --rm --network host alpine ip -4 -o addr show | grep -v ' lo '
podman run --rm --network host nginx:alpine sh -c 'nginx; sleep 1' 2>&1 | tail -1
podman run --rm --network none alpine wget -qO- --timeout=2 http://1.1.1.1 2>&1 | tail -1
podman run --rm --network none alpine ip -4 -o addr
podman run --rm alpine sh -c 'getent hosts host.containers.internal; wget -qO-
http://host.containers.internal:18080/api/actuator/health' ; echo
```

**Observeer** je eigen WSL-interfaces met `--network host`, dan `nginx: [emerg] bind() to 0.0.0.0:80 failed (13: Permission denied)`; met `--network none`, `wget: can't connect to remote host (1.1.1.1): Network unreachable` en alleen `lo`; tot slot `169.254.1.2 host.containers.internal` en `{"status":"UP"}`: vanuit het standaardnetwerk zijn de host en zijn gepubliceerde poorten onder die naam bereikbaar.

*Uitleg.* `host` = geen isolatie (en de beperkingen van jouw gebruiker); `none` = volledige isolatie; `host.containers.internal` = "de machine waarop ik draai", het Podman-equivalent van `host.docker.internal`.

---

## Stap 9 — Dezelfde keten in een pod

```
podman pod create --name stackpod -p 18090:80
podman run -d --pod stackpod --name api-pod api:1.0
podman run -d --pod stackpod --name web-pod -v ./nginx-
pod.conf:/etc/nginx/conf.d/default.conf:ro nginx:alpine
sleep 2 ; podman pod ps
podman ps --pod --format 'table {{.Names}}\t{{.Pod}}\t{{.Ports}}'
curl -s localhost:18090/api/actuator/health ; echo
podman exec api-pod cat /etc/hosts | tail -2
podman pod rm -f -t 0 stackpod
```

**Observeer** een pod `Running` met `3` containers (de derde is de *infra*); in `ps --pod` delen `api-pod` en `web-pod` dezelfde `POD ID` en dezelfde publicatie `0.0.0.0:18090->80/tcp`, gedragen door `…-infra`; `{"status":"UP"}` via een nginx die doorstuurt naar `localhost:8080`; en in `/etc/hosts`, `127.0.0.1 api-pod` en `127.0.0.1 web-pod`.

*Uitleg.* In een pod is er één network namespace voor iedereen: `localhost` verbindt de containers, en het is de pod die publiceert. Je hebt net, zonder het te weten, het equivalent van een Kubernetes-Pod met twee containers uitgevoerd.

---

## Opruimen

```
podman network rm stack
podman rm -f -t 0 web api db
podman network rm stack
podman network ls
podman rmi docker.io/library/postgres:16-alpine
```

**Observeer** eerst `Error: "stack" has associated containers with it. Use -f to forcibly delete containers and pods: network is being used`, en daarna, na het bij naam verwijderen van de drie containers, `stack` verwijderd en alleen `podman` over. Hou `api:1.0` en `~/labo-docker/07/`: labs 08 en 09 hergebruiken de image en de nginx-bestanden.

---

## Wat je nu moet kunnen beweren

- Standaard rootless-netwerk = `pasta`: leeg `IPAddress`, het adres van de host gekopieerd, geen naam.
- Netwerk `podman`: IP's `10.88.0.x`, `ping: bad address` op naam; eigen netwerk: `dns_enabled: true`, `getent hosts api` antwoordt.
- `EXPOSE` publiceert niets: `podman port api` leeg; `-p 18080:8080` publiceert; `-P` kiest een willekeurige poort; `-p 80:…` → `permission denied` in rootless.
- nginx stuurt `/api/` door naar `api:8080` op naam; één origin, geen CORS.
- API opnieuw aangemaakt = nieuw IP = `502` bij nginx, `200` na `restart web`.
- `--network host`: jouw interfaces en jouw beperkingen; `--network none`: `Network unreachable`.
- Een pod deelt `localhost` en publiceert zelf: `3` containers waarvan de infra, `127.0.0.1 api-pod`.